

Fuzz4Cuda:Fuzzing Your NVIDIA GPU Libraries Through Debug Interface

Yuhao Zhou^a, Peng Jia^a, Jiayong Liu^a and Ximing Fan^a

^aSichuan University, Jiang'an Campus, No. 2, 2nd Section, Chuanda Road, Xihanggang Street, Shuangliu District, Chengdu, 610207, China

ARTICLE INFO

Keywords:
Fuzzing test
GPU library
Debug interface
Streaming Assembler analysis

ABSTRACT

The programming security of Compute Unified Device Architecture(CUDA), NVIDIA's parallel computing platform and programming model for Graphics Processing Unit, has always been a significant concern. On the host-side, fuzzing has been remarkably successful at uncovering various software bugs and vulnerabilities, with hundreds of flaws discovered annually through different fuzzing tools. However, existing fuzzing tools typically operate on general-purpose CPU architectures and embedded system. As an independent processing unit, the GPU does not support tools like American Fuzzy Lop for collecting instrumentation and code coverage information. Consequently, gray-box fuzzing for closed-source graphics and driver libraries has remained an unaddressed challenge. This research introduces Fuzz4Cuda, a universal fuzzing framework specifically designed for GPU libraries. To enhance device-side coverage collection, Fuzz4Cuda achieved this by runtime analysis of CUDA Streaming Assembler. Furthermore, the framework could dynamically adjust the number of breakpoints to optimize test case execution speed, thereby accelerating the overall time to discover program crash inputs. The development of Fuzz4Cuda has moved GPU library fuzzing ahead, aiming to improve the security of the GPU programming environment. During a month-long experimental period, our evaluation of the CUDA Toolkit library led to the discovery of five real-world bugs, four of which have been assigned CVE IDs.

1. Introduction

Fuzz testing[33, 3, 16, 27, 10, 5, 31, 25], an automated technique, involves inputting unexpected, random, or semi-random data into a target system and monitoring for anomalous behaviors like crashes or memory leaks to uncover vulnerabilities. It has become an indispensable tool in software security, widely applied across operating systems, network protocols, browsers, and embedded systems. Extensive research confirms the significant effectiveness of fuzz testing in vulnerability discovery, with existing fuzzing tools successfully detecting numerous security flaws in real-world systems.

Traditional fuzzers, such as Google's AFL++[11], LibFuzzer [18], Hopper[4], Utopia[13], primarily serve software programs running on CPU. Unfortunately, these well-known fuzzers are largely ineffective at fuzzing applications that utilize GPU, presenting several limitations and challenges:

C1: GPU kernel functions have limited support for CPU libraries and system calls due to architectural differences. GPU-executable functions, known as kernel functions, are written using C-like syntax but do not support the vast majority of CPU-specific library functions or system calls. This is due to GPU possessing specialized architectures and instruction sets, such as NVIDIA's CUDA[20] architecture with its corresponding Streaming Assembler (SASS), and AMD's ROCm[1] architecture, which is even more complex with three distinct instruction set architectures: GCN (Graphics Core Next), CDNA (Compute DNA), and RDNA (Radeon DNA). System call code written for x86 or ARM instruction sets is largely incompatible with these GPU instruction sets.

Even fundamental print functionalities like printf were only natively supported in CUDA 6.0 and later, making traditional fuzzer instrumentation methods unsuitable for GPU kernel code.

C2: Current fuzzing tools[15] can not observe or collect coverage from the GPU side, making coverage-guided fuzzing for GPU code infeasible. This limitation arises because the specialized architecture of GPUs prevents binary analysis tools such as QEMU from handling GPU-executed code effectively. QEMU primarily emulates CPU instructions and, in the context of AFL++, enables fuzzing by executing binaries without instrumentation conflicts while maintaining high throughput. However, it lacks support for GPU-specific architectural instructions and can not emulate the execution of GPU kernels, making it impossible to track code coverage on the GPU side. In addition, QEMU can not accurately simulate CPU-GPU collaborative execution, which is essential for many GPU workloads but uncommon in traditional CPU-only programs.

C3: GPU initialization incurs high overhead due to context management and real-time compilation. For every program that invokes a GPU, the GPU must maintain a context, and the Parallel Thread Execution (PTX) code of the program requires just-in-time (JIT) compilation on the GPU before execution. Unlike CPU binaries, which are pre-compiled and ready for immediate execution, GPU kernels require just-in-time compilation and context initialization before execution. This cost is further amplified in AFL++'s persistent mode, which relies on the fork mechanism, as each forked process triggers a new GPU initialization and resource setup. Preliminary measurements show that each initialization incurs a cost of approximately 0.7s for a cold

*Corresponding author: Ximing Fan (fanximing@stu.scu.edu.cn)
ORCID(s):

start and 1.6s for a warm start, resulting in a prohibitive delay for high-throughput fuzzing.

Our Work. To address the unique challenges in fuzzing GPU libraries, we propose **Fuzz4Cuda**, a universal and efficient fuzzing framework tailored for the security and stability analysis of CUDA-based GPU libraries. Unlike existing approaches, Fuzz4Cuda supports closed-source library fuzzing without requiring manual stub generation. Our contributions are threefold:

Kernel Control Flow Graph Construction. To enhance GPU kernel coverage detection for fuzzing, we designed a framework that dynamically constructs Control Flow Graphs for CUDA kernels and uses them to guide the fuzzing process. While existing tools such as NVBit[28] provide instrumentation capabilities, they can not extract kernel CFGs directly. Although static tools like `nvdiasm` can obtain CFG via disassembly, they lack coverage feedback and thus can not support runtime-guided fuzzing. To the best of our knowledge, this is the first work to leverage GPU kernel coverage information to guide the fuzzing process. Our approach overcomes the generality limitations of prior methods and enables effective tracking and analysis of kernel execution paths.

Persistent Mode. To significantly mitigate the high overhead associated with GPU initialization, Fuzz4Cuda integrates a persistent fuzzing strategy directly within the CUDA-GDB environment. This approach enables the reuse of GPU resources across multiple test cases, thereby bypassing the repetitive GPU initialization inherent in traditional fork mechanisms. By maintaining a persistent fuzzing process, our framework substantially boosts testing throughput.

Optimization for Closed-Source Libraries and Large Basic Blocks. Computational libraries provided by the CUDA Toolkit are typically closed-source. Consequently, static linking of the entire library during compilation leads to an exceptionally large number of basic blocks analyzed by tools like Ghidra, and we lacked technical means to filter out irrelevant function symbols. Furthermore, an excessive number of software breakpoints can severely impede fuzzing execution speed. Given the immense number of basic blocks in CUDA libraries, we've developed a dynamic adjustment algorithm based on the balance between breakpoint count and execution speed. This algorithm intelligently optimizes code coverage and test execution speed, maximizing fuzzing effectiveness under limited resources.

2. Background

2.1. Grey-Box Fuzzing test

Fuzz testing is an effective automated testing technique that feeds malformed or unexpected inputs into target systems to uncover anomalous behaviors and potential vulnerabilities. Over the years, it has become a cornerstone of software security, widely adopted across operating systems, protocol stacks, web browsers, embedded systems, and various applications.

Grey-box fuzzing, as a central paradigm in modern vulnerability discovery, has consistently revolved around the

core proposition of optimizing testing efficiency with limited feedback. Unlike black-box fuzzing, which relies solely on observing inputs and outputs, grey-box fuzzing leverages lightweight instrumentation to gather partial internal state information of program execution flow. This approach skillfully avoids the immense computational overhead required by white-box fuzzing while effectively providing guidance feedback, which significantly boosts testing efficiency.

This compromise methodology of grey-box fuzzing can be traced back to the DART framework proposed by Godefroid[12], which combined symbolic and concrete execution to lay early foundations for path exploration. However, the true breakthrough in this domain occurred in 2014 with the advent of AFL, developed by Michal Zalewski. AFL pioneered the integration of branch coverage feedback with genetic algorithms. By continuously selecting and mutating test cases that trigger new execution paths, AFL improved vulnerability discovery efficiency by two to three orders of magnitude, tremendously propelling the development and widespread adoption of fuzzing technology.

Nevertheless, mainstream grey-box fuzzing tools like AFL++ and LLVM's LibFuzzer mandate recompilation of the target program to inject coverage tracking logic. This presents severe practical limitations when dealing with closed-source software or complex and hard-to-modify build environments.

To overcome this constraint, the industry has gradually seen the emergence of binary-level grey-box fuzzing, achieving remarkable progress. These methods do not require access to program source code; instead, they efficiently discover vulnerabilities directly through binary analysis and lightweight instrumentation techniques. For instance, the LLVM compiler can transform program source code into LLVM Intermediate Representation and perform instrumentation at this intermediate layer. However, due to the considerable differences between CPU and GPU, the current Clang compiler does not support instrumentation of GPU intermediate representation languages, including PTX used by NVIDIA. Although AMD recently introduced simultaneous AddressSanitizer insertion for CPU and GPU to detect memory errors, the feedback generated can not be fed back to the fuzzer for code coverage.

2.2. Debugger Interface

The GNU Debugger (GDB) is a core tool in binary analysis and software debugging. Its power lies in its ability to perform fine-grained analysis and control over program execution at the lowest levels. As an open-source debugger, GDB is renowned for its exceptional flexibility and broad compatibility. It supports a multitude of CPU architectures and numerous programming languages, making it an indispensable asset for developers and security researchers working with compiled binaries. Functionalities of GDB are extensive, ranging from basic execution control to advanced memory and register inspection, allowing users to inspect and modify the program's internal state in real time. When symbol files are available, GDB can perform both

disassembly-level debugging and source-level debugging, effectively bridging the gap between high-level programming constructs and low-level machine implementations, thus providing a comprehensive perspective on program behavior.

Within GDB, breakpoints are the core mechanism for interrupting program control flow. When program execution reaches the location of a breakpoint, it automatically pauses, allowing the debugger to take control.

Given the unique nature and complexity of GPU parallel computing, major graphics card manufacturers have developed customized debugging tools based on the open-source GDB framework. These tools are designed to provide deep support for their respective GPU architectures and programming models. This approach effectively avoids potential discrepancies introduced by simulation or emulation environments, thereby ensuring the accuracy and reliability of debugging results. NVIDIA CUDA-GDB is an integral part of the CUDA toolchain. It supports SASS instruction-level debugging for CUDA kernel functions, enabling developers to inspect low-level instruction execution details on the GPU. Furthermore, it detects out-of-bounds global memory accesses, which is crucial for identifying common memory errors in GPU programs.

2.3. CPU Architecture and GPU Architecture

The evolution of modern computing systems is profoundly influenced by their underlying processor architectures. Central Processing Units and Graphics Processing Units employ distinctly different design philosophies and execution models, optimized for serial and parallel workloads, respectively.

CPU Architecture: x86 is the most widely used architecture in general-purpose computing. It follows the CISC paradigm, allowing complex operations to be performed with single, multi-cycle instructions. Contemporary x86 processors incorporate advanced features such as out-of-order execution, deep instruction pipelines, and SIMD extensions (e.g., AVX-512). Its dominance in general-purpose computing is largely attributed to extensive backward compatibility and high single-threaded performance. The CPU executes the operating system kernel at the highest privilege level and provides system call mechanisms for user-space programs to access kernel services such as memory management and process scheduling.

GPU Architecture and System Interaction: CUDA is the parallel computing platform and programming model developed by NVIDIA for general-purpose GPU computing. It leverages the architecture of GPU, which execute thousands of lightweight threads in parallel to perform the same instructions on different data. RDNA is the modern GPU architecture introduced by AMD. Although it is primarily optimized for graphics rendering, RDNA also supports general-purpose computing through programming interfaces such as OpenCL and HIP.

In contrast to CPU, GPU is designed as specialized accelerators with a focus on parallel computation. Their hardware

architecture simplifies control logic and does not implement operating system privilege levels. The instruction sets of GPU mainly consist of arithmetic and memory operations and do not include instructions capable of directly invoking kernel-mode system calls.

From the perspective of system interaction, system calls on the CPU involve a transition into kernel mode, typically through a system call instruction or software interrupt. By comparison, GPU does not interact with the operating system directly. Instead, the CPU submits computational tasks to the GPU through device drivers, which serve as intermediaries. Additionally, GPU memory is physically separated from system memory. The operating system does not directly manage GPU memory; instead, all memory-related operations are mediated by the driver.

These architectural characteristics impose significant challenges for implementing execution coverage collection on GPU. Traditional coverage-guided fuzzers, such as AFL++, rely on compile-time instrumentation to insert code that updates bitmap data based on block-level hash values. Applying similar techniques to GPU code requires modifying original function signatures to pass additional data between the GPU and CPU. This approach is not feasible when dealing with pre-compiled or closed-source libraries. Therefore, new runtime mechanisms must be developed to enable effective fuzzing in such black-box settings.

```

1 0x00 MOV R1, c[0x0][0x28]
2 0x10 MOV R1, R0;
3 0x20 SETP.EQ.U32 P0, R0, 0x0;
4 0x30 @P0 BRA 0x200;
5 //if P0 is 1,branch(jump) to 0x200
6 0x40 MOV R1, R2;
7 .....
8 0x200 BSSY B0, 0x400;
9 .....
10 0x230 @P0 BREAK B0;
11 //if P0 is 1,break barrier and jump to 0x400
12 .....
13 0x380 BSYNC B0;
14 //wait for other thread and jump to 0x400
15 .....
16 0x590 EXIT
17 0x600 NOP
18 0x610 NOP
19 .....
20 //A lot of NOP

```

Listing 1: Example Code of Streaming Assembler

SASS is the low-level instruction set architecture for NVIDIA GPU, representing the machine code instructions that CUDA programs ultimately execute on the GPU after compilation. It is analogous to assembly language in traditional CPU architectures and operates as a lower-level representation beneath PTX, which functions as a virtual instruction set architecture. SASS is tightly coupled with specific GPU hardware architectures. According to NVIDIA's official "CUDA Binary Manual" and our practical experience,

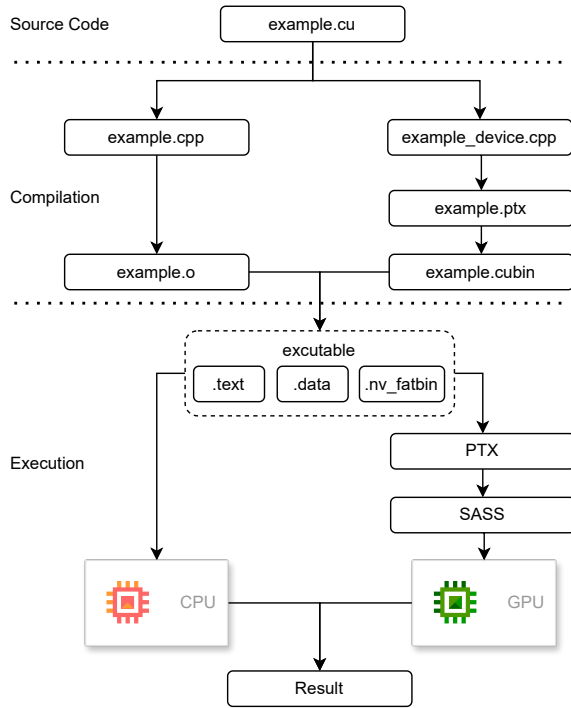


Figure 1: GPU Program Compilation and Execution Pipeline

the SASS instruction set also includes control flow instructions similar to `cmp`, `jmp`, `jz`, and `jnz`. In SASS code, the SETP instructions like `CSETP` and `PSETP` are used to set the values of predicate registers within the GPU. These predicates, when combined with `BRA`, enable conditional jumps. As shown in Listing 1, `0x100 !@P1 BRA 0x560` means that if predicate `@P1` is false, the program will jump to address `0x560`.

CUDA Program Compilation and Execution: As illustrated in Figure.1, during the compilation of a CUDA program, compilers like `nvcc` separate and compile GPU code and CPU code based on function decorators like `host` and `device`. The right side of the Figure.1 depicts the specific aspects of CUDA program compilation. `Nvcc` first compiles the GPU portion of the code into PTX, which is then merged into the `nv_fatbin` section of the ELF file during the linking phase. Subsequently, during execution, Just-In-Time (JIT) compilation occurs, wherein the PTX code stored in the ELF file is compiled into the actual SASS code that executes on the GPU device. Finally, the collaborative execution process between the CPU and GPU begins, and the program completes its execution.

3. Related work

3.1. Fuzz Testing for GPU Libraries

Most existing research and tools[32, 2] focus on either fuzzing high-level deep learning frameworks or web shaders, while testing efforts rarely target the underlying C/C++ implementations or low-level libraries that interact directly with GPU drivers. Despite significant advancements in binary-level fuzzing within the CPU domain, extending this paradigm to low-level GPU-related libraries introduces unique complexities and substantial challenges, resulting in very limited fuzzing efforts specifically targeting GPU libraries.

Tools like `Lemon`[29], `FreeFuzz`[30] and `DeepREL`[8] focus on the Python interface layers of `PyTorch` and `TensorFlow`. They typically use deep learning model to automatically generate input parameter combinations and API call sequences. Tools like `TitanFuzz`[6] and `FuzzGPT`[7] take it a step further, leveraging large language models to learn complex relationships between APIs, then executing fuzz tests via Python-level interfaces. These methods often build test cases based on call documentation or API flow graphs, and they rely on Python's exception mechanisms or inconsistencies between CPU and GPU execution results to identify potential flaws. However, these methods run entirely within the Python interpreter and severely lack coverage information for actual low-level GPU execution. This means their fuzz testing only covers the upper-level encapsulation logic and intermediate components, making it hard to effectively uncover deep vulnerabilities at the GPU hardware or driver levels. This limitation is exactly what our research aims to address.

3.2. Fuzz Testing accross CPU

Beyond vulnerabilities purely residing in GPU libraries and drivers, the tight collaboration between CPUs and specialized processing units, such as GPU and NPU, in modern heterogeneous computing systems introduces unique security challenges. This multi-core collaboration, facilitated through shared memory or specific communication interfaces, brings inherent complexity that makes fuzzing more difficult, yet also points to potential attack surfaces. Device drivers, acting as the bridge between the operating system kernel and hardware devices, typically access hardware resources directly. This means any vulnerability within a driver could potentially lead to system crashes, data leaks, or even arbitrary code execution, posing a severe threat to system security [19]. Especially in the context of tight cross-processor collaboration, this design, while significantly boosting performance, unfortunately also introduces entirely new attack surfaces. A major challenge for current low-level fuzzing, particularly cross-processor fuzzing, is effectively identifying the attack potential of shared memory regions and generating precise payloads to trigger vulnerabilities[34]. `Moneta` [15] uses an ex-vivo technique to simulate GPU contexts, allowing fuzz testing

without a physical GPU, which significantly boosts efficiency. However, this method can't uncover potential GPU-specific issues, such as a GPU becoming unresponsive due to incorrect parameters. To address the challenges arising from complex CPU-XPU interactions and to fill the research gap in these novel attack surfaces, our research aims to deeply explore fuzzing methodologies that can effectively cover low-level GPU execution paths while also considering the security of multi-core collaboration.

3.3. Algorithmic Testing for GPU Kernels

Previous studies[23, 24] introduced a test generation technique for OpenCL kernels. This method combines mutation-based fuzzing with selective constraint solving. Unlike traditional fuzz testing, which primarily aims to uncover security vulnerabilities, the main goal of this technique is to help identify data races between work-groups.

3.4. Fuzz Testing for GPU Compilers

Fuzz testing in this domain[14, 17] is relatively simpler than the other Three. Compilers do not need to care about the current GPU device state. They only need the architecture of target graphics card device to perform compilation. Technically, it's quite similar to other compiler fuzzing approaches.

3.5. Fuzzing Testing for Embedding System

GDBFuzz[9] addresses several challenges in traditional embedded hardware fuzzing by circumventing the complexities of full hardware virtualization. Conventional embedded hardware fuzzing encounters obstacles such as the substantial maintenance costs associated with virtualizing instructions and devices, which necessitate continuous updates and intricate configurations for accurate hardware behavior simulation. Furthermore, the inherent difficulty, and often impracticality, of virtualizing embedded devices featuring multi-core or dual-CPU architectures presents a formidable barrier. These complex architectural designs involve intricate inter-processor communication and shared resource management, which are not readily amenable to conventional virtualization techniques, potentially leaving gaps in security testing. Although GDBFuzz is not specifically designed for CUDA fuzzing and can not effectively fuzz CUDA libraries, it provides a general framework capable of fuzzing binary executables. The original GDBFuzz can perform fuzz testing on binaries but lacks specialized support for GPU environments. Nonetheless, its approach to managing challenges related to multi-processor collaboration has offered valuable inspiration and laid a foundational basis for our work.

4. Design

Inspired by the concepts in GDBFuzz, we utilize CUDA-GDB to obtain control flow graphs and introduce targeted enhancements to overcome the unique challenges of GPU fuzzing. As shown in Figure.2, the process begins with the extraction of the CFG from the target executable, after which the executable is launched within CUDA-GDB and initial

breakpoints are configured. Test inputs are then provided to the running program. If no breakpoint is triggered, the input is considered uninformative and any resulting crash is recorded. Concurrently, the system continuously monitors and records the execution speed in relation to the active breakpoint count. This data is utilized to construct a performance curve, which in turn informs the selection of an optimal breakpoint quantity to maximize fuzzing efficiency. Conversely, when a breakpoint is triggered, Fuzz4Cuda will re-monitor the launch of additional CUDA kernels. If a previously unseen kernel is launched, the corresponding kernel CFG is subsequently extracted, and coverage information is recorded. Based on the collected feedback, new test inputs are generated to guide further exploration. The executable is restarted only when the program crashes or when the number of iterations in persistent mode reaches a predefined threshold, thereby enabling iterative and efficient exploration of kernel execution paths to maximize coverage and identify potential vulnerabilities.

4.1. Persistent Mode for GPU Program

Persistent mode needs to be accomplished through the collaboration of the fuzzer and the harness. Persistent mode allows us to address C3, which is that programs on the GPU have a larger initialization overhead compared to programs on the CPU.

Suppose we currently want to test the `nvjpegDecode` function within the `nvjpeg` library of the CUDA Toolkit. We need to reduce the overhead caused by program startup by reusing some resources and sharing a single GPU context across multiple tests.

```

1 #include <cuda.h>
2 #define GDB_LOOP 100000
3 void hook(){}
4 void error_check(){//user defined}
5 int main(int argc, char* argv[]){
6     init_resource();
7     for(int i=0;i<GDB_LOOP;i++){
8         hook();
9         //GDB will stop here everytime
10        //and wait for input
11        read_input();
12        init_in_loop_resource();
13        return_value=target_API();
14        error_check(return_value);
15        free_in_loop_resource();
16    }
17    free_resource();
18 }
```

Listing 2: Example of Persistent Mode Harness

As shown in Listing 2, we need to insert an empty hook function. The purpose of this function is to provide a breakpoint in the program. Each time CUDA-GDB runs through a loop, the program will pause at the hook function and wait for a new input.

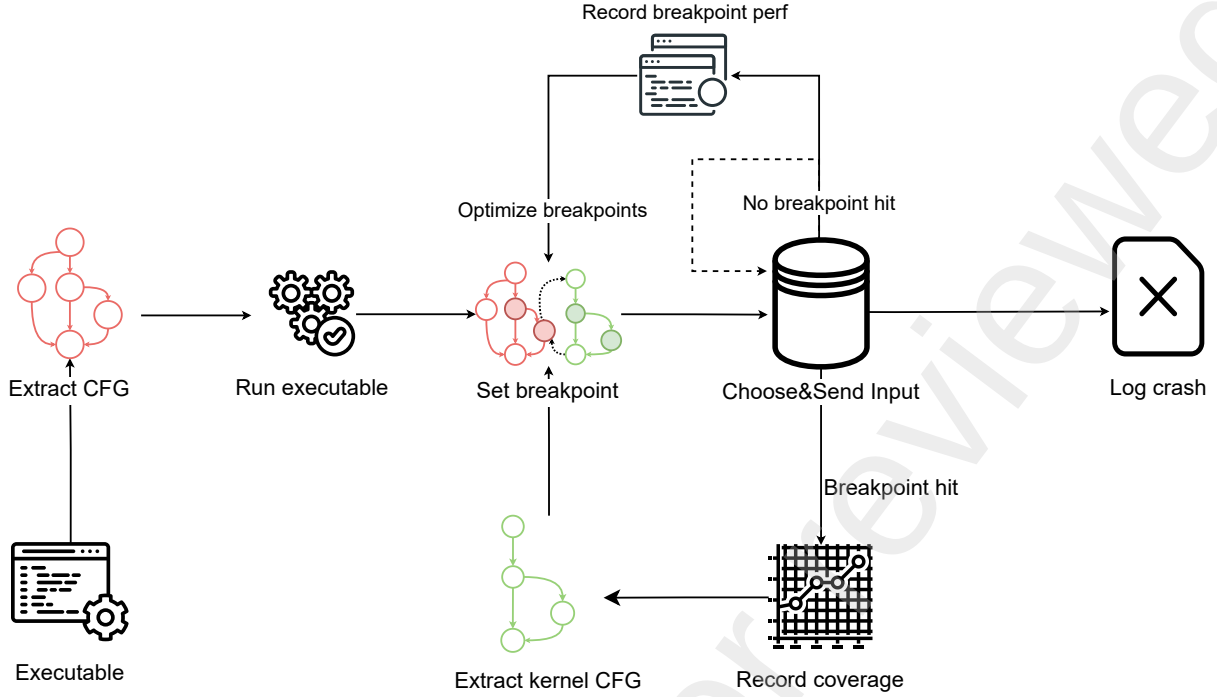


Figure 2: Framework for Fuzz4Cuda.

4.2. CFG of SASS Instructions

Building upon our understanding, BRA, EXIT, RET, BREAK, and BSYNC are defined as the termination instructions of a basic block. A key distinction from x86, where comparison and jump instructions are usually at separate addresses, is that in GPU, predicates and the instructions can share the same address. For predicated instructions, two distinct approaches were initially considered. The first approach posited that the predicate instruction itself does not alter control flow, given that program execution invariably reaches its address. The second approach involved treating each predicated instruction as an independent basic block. Ultimately, the former approach was adopted due to its alignment with nvdiasm's handling of predicates.

During basic block connection, if a branching control instruction appears without a predicate modifier, no fall-through edge is connected. After connecting fall-through edges and BRA jump edges, special handling is required for BREAK and BSYNC instructions. These instructions typically appear in pairs with BSSY instructions to define synchronization points. For instance, BREAK B0 or BSYNC B0 will jump to the address associated with label B0 defined by a preceding BSSY B0, <target_addr>. Therefore, when constructing the CFG, we need to traverse forward to find and connect to the jump address of the most recently corresponding BSSY instruction.

Furthermore, inspired by previous GDBFuzz work, we also applied a strict dominance relation strategy to the basic

blocks of SASS code. While its effect might not be pronounced in scenarios with fewer basic blocks and numerous breakpoints, for GPU programs, which often have an enormous number of basic blocks, strict dominance relations provide crucial structural information. The strict dominance relation strategy can be expressed by the following relationships, as shown in Equation 1. For two distinct nodes d and n in a control flow graph, d strictly dominates n if every path from the entry node to node n must pass through node d .

$$d \triangleright n \iff \begin{cases} d \neq n, \\ d \in \bigcap_{p \in \text{Paths}(\text{entry}, n)} p \end{cases} \quad (1)$$

In our implementation, we first resolve the actual offset of instructions. Then, we identify the starting points of basic blocks, which include function entry points and the target addresses of various control flow transfer instructions, as well as the fall-through paths of conditional branches.

Next, we partition the individual basic blocks based on the identified leaders. Subsequently, the code iterates through each basic block and establishes control flow edges between them based on the type of its last instruction and the presence of predicates. For BREAK and BSYNC instructions, their corresponding BSSY definitions are traced back to connect them to the correct jump targets.

Finally, we compute the dominator sets for all basic blocks. This computation is performed based on the definition of strict dominance as given in Equation 1. Applying this definition to the constructed basic blocks and edges, we obtain the complete SASS code control flow graph and its dominance relationships.

4.3. Runtime SASS CFG Analysis

As mentioned in the previous Background Section 2.3, code on the GPU is not directly stored in the ELF file. Instead, during compilation, the compiler translates the code into PTX code, and temporarily stores this compiled PTX code in the ELF file, for example, in the `nv_fatbin` section. This is because a CUDA program might be executed on GPU with different architectures. Subsequently, when the program runs, the PTX code stored in the ELF file is again compiled into the SASS code that actually executes on the GPU. This means that during the reverse engineering analysis phase of a program, we can not obtain the actual runtime addresses of the code on the GPU. This leads to the necessity of designing a process to analyze GPU execution instructions in real-time at runtime.

Combining this with the persistent mode approach from Section 4.1, we need to perform CFG analysis of SASS code at runtime. Our design is as follows: In the initial few rounds of execution, we set the `cuda_break_on_launch` all parameter to interrupt all incoming kernel functions and obtain their instructions. Subsequently, we set it to none to avoid the overhead of repeated interruptions at kernel functions. However, this approach has some drawbacks. A single seed might not necessarily trigger all kernel functions. To compensate for this shortcoming, we supplemented with another strategy: after a basic block belonging to the host side execution is triggered, we re-enable the interrupt upon entering the GPU. The reason for this is that in CUDA, programs need to use specific syntax to launch GPU kernel functions, as shown in the following Listing 3. If a new kernel function is triggered, a new block must be reached.

```

1 // Kernel function definition
2 __global__ void add(int *a, int *b, int *c) {}
3 int main(){
4     int a[10], b[10], c[10];
5     int *d_a, *d_b, *d_c;
6     ...//Memory Malloc
7     //Kernel Launch
8     add<<<1, 10>>>>(d_a, d_b, d_c);
9     ...//Memory Free
10    return 0;
11 }
```

Listing 3: Syntax of Kernel Launch

4.4. Optimizing Breakpoint Strategy

Our approach directly addresses a key challenge we encountered while fuzzing closed-source GPU libraries. In the GPU environment, the combination of handling complex scenarios and using Ghidra for static library linking—needed to build a complete control flow graph of

CUDA library functions—results in an enormous number of basic blocks in the target programs.

Take `cusolver`, a library for matrix computations, for example. The program under test can have nearly 56,000 blocks. At this massive scale, generating program dominator relationships can extend to roughly 30 minutes. With a substantial number of basic blocks and a restricted allocation of breakpoints, block coverage demonstrated an almost linear increase over time. However, as more breakpoints were introduced, their marginal utility diminished, with each additional breakpoint contributing progressively less to overall coverage. Concurrently, an excessive number of breakpoints significantly burdened CUDA-GDB, leading to an overload of breakpoint information and a considerable reduction in fuzzing execution speed.

To effectively ease this critical issue, we came up with a method: we fit a curve based on the relationship between the number of breakpoints and program execution speed, then set a speed drop threshold to limit the breakpoint count. Our main goal is to maximize code coverage while keeping execution speed as fast as possible.

We used an exponential decay model for our fitted curve. By building this precise model, we can quantify the impact of breakpoint count on performance, which stops us from blindly adding breakpoints during fuzzing and suffering sharp performance drops for little gain. The performance model is defined as:

$$\text{Performance}(x) = a \cdot e^{-b \cdot x} + c \quad (2)$$

where x represents the number of breakpoints, and $\text{Performance}(x)$ is the expected execution speed at x breakpoints. Here, a is a constant signifying the maximum theoretical performance as the breakpoint count approaches zero. The decay coefficient b dictates the rate at which performance decreases with an increasing number of breakpoints, while c acts as an offset or asymptote, indicating the minimum performance the system approaches under very high breakpoint counts.

After fitting this performance curve, we set a clear speed drop threshold. This ensures that any increase in breakpoint count, aimed at enhancing code coverage, does not result in an unacceptable reduction in execution speed beyond this preset limit. This strategy avoids the trap of reducing testing efficiency by chasing coverage at all costs. This method enables optimal fuzzing effectiveness and practical utility under limited computing resources and testing time.

5. Experiments Analysis

5.1. Experimental Setup

Target libraries. We primarily evaluated the `nvjpeg` library included in the CUDA toolkit [22], focusing on its encoding and decoding functions. According to NVIDIA's official CVE disclosures, `nvjpeg` has historically contained security vulnerabilities. To further assess the generality of Fuzz4Cuda, we additionally generated fuzzing harnesses for

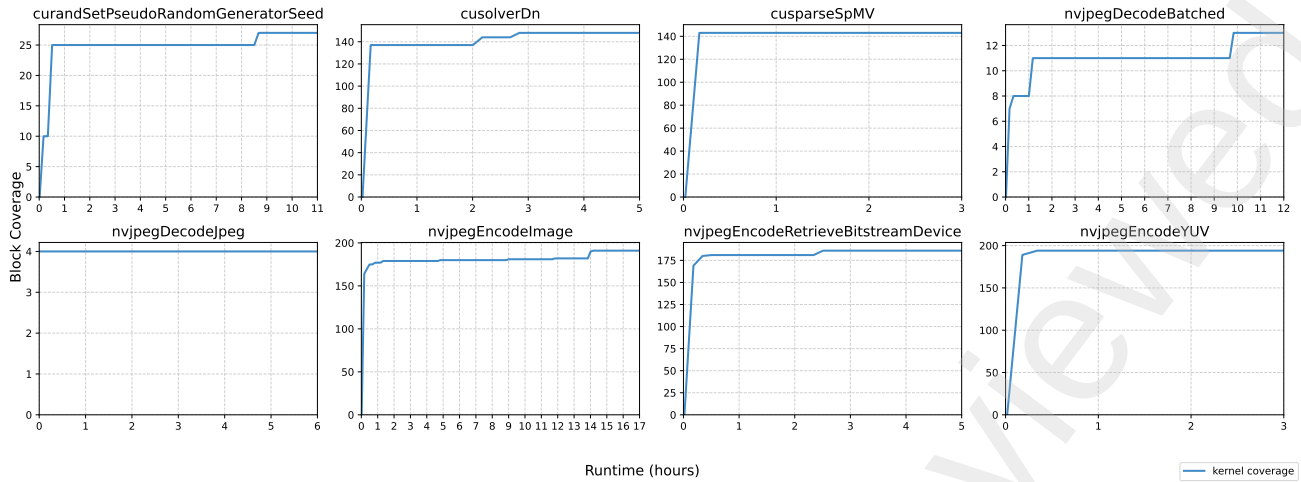


Figure 3: Kernel Coverage over 24 Hours for Eight Different APIs

several other NVIDIA libraries [21], including cuFFT, cuRAND, cuFile, cuSolver, CUDA Driver, CUDA Runtime, CUDA Math, and NPP.

Hardware environment. All experiments were conducted on a workstation equipped with an AMD Threadripper Pro 5995WX CPU and two NVIDIA RTX A6000 GPUs. Each GPU was exclusively allocated to a single fuzzing task at any given time.

Software environment. The experimental environment was based on NVIDIA's official Docker image with CUDA version 12.4. We used Fuzz4Cuda integrated with libFuzzer as the fuzzing framework. Basic block coverage was collected separately for both the host and device sides.

5.2. Metrics

We evaluate Fuzz4Cuda from the following aspects.

- **RQ1:** What is the basic block coverage achieved on the GPU side?
- **RQ2:** Can Fuzz4Cuda Discover Real-World GPU Vulnerabilities?
- **RQ3:** How does Fuzz4Cuda compare against existing fuzzers? To what extent is Fuzz4Cuda effective in discovering real-world bugs in GPU applications?
- **RQ4:** How effective is persistent mode in improving the speed of GPU fuzz testing?
- **RQ5:** How effective is the strategy of using a speed drop threshold to control the number of breakpoints?

5.3. RQ1: Kernel Block Coverage

For a long time, effectively detecting and leveraging kernel code coverage has been a challenge for traditional fuzzing tools in the realm of GPU fuzzing. These conventional tools often struggle to penetrate the kernel execution flow beneath the GPU hardware abstraction layer, let alone precisely measure and guide the fuzzing process to explore deeper.

For kernel block coverage, we selected a set of kernel APIs with relatively complex functionality, along with a small number of simpler APIs, as representative cases for illustration. Each harness was executed for a total of 24 hours to ensure the comprehensiveness and stability of the results. However, for better visualization of the data, our figures do not cover the entire 24-hour experimental period. Instead, the x-axis is truncated at three hours after the last observed increase in coverage.

In RQ2, when the number of basic blocks in the code under test was relatively small, dynamic breakpoint strategies do not significantly improve code coverage. This meant that for simpler or smaller code snippets, dynamically adjusting breakpoint locations do not yield substantial performance gains. Based on this finding, and to more precisely and clearly evaluate the actual performance of our proposed Fuzz4Cuda tool in specific, more challenging scenarios, we strategically employed a fixed breakpoint strategy in this experiment. We uniformly set the number of breakpoints to $n = 10$, aiming to exclude any additional variables that dynamic breakpoint strategies might introduce, thus allowing us to more purely focus on Fuzz4Cuda's core capabilities in basic block coverage data collection and guidance.

As illustrated in Figure 3, the experimental results demonstrate Fuzz4Cuda ability to leverage kernel basic block coverage for fuzzing. Fuzz4Cuda improved coverage of existing code paths and successfully explored previously unreached kernel code basic blocks. For instance, in the case of the nvjpegEncodeImage API, Fuzz4Cuda continued to discover new coverage even after 14 hours of fuzzing. This sustained increase in coverage demonstrates its ability to perform long-term and in-depth exploration of complex GPU kernels. This deep exploration facilitates the discovery of potential vulnerability areas that traditional fuzzing tools were previously unable to access. This advancement offers new approaches for fuzzing GPU kernel code and enhances the depth and efficiency of the overall testing process, contributing to the security and reliability of GPU software.

Table 1
CVE Assignment

CVE ID	Tracking Number	Vulnerability Type
CVE-2025-23272	5168024	Null Pointer Dereference
CVE-2025-23273	5218311	Unexpected Division by Zero
CVE-2025-23274	5216452	Out of Bound GPU Memory Access
CVE-2025-23275	5225718	Denial of Service Due to GPU Memory Allocation Error
-	5339845	Unexpected Division by Zero

Table 2
Comparison of CVE Discovery Times

Traking Number	Fuzz4Cuda	AFL++(none instrument)
5168024	7s	371s
5216452	10s	-
5218311	15h08m	-
5225718	10h39m	-
5339845	14s	470s

5.4. RQ2:Real World Bug

During our month-long fuzz testing of the nvjpeg library, we achieved significant results, successfully discovering multiple vulnerabilities. These vulnerabilities have been assigned four CVE numbers, with details provided in Table 1.

One of these errors occurred within the GPU kernel, while another caused a system-wide Denial of Service due to the GPU device. Notably, this vulnerability can cause the entire system to hang—even when executed inside a Docker container, it may still lead to the host machine becoming unresponsive. This shows that Fuzz4Cuda could delve deep into the kernel and uncover potential security issues within GPU devices.

5.5. RQ3:Comparison with existing fuzzing tools

In this section, we aim to conduct a comparison of the performance of a mainstream fuzzing tool, AFL++ with Fuzz4Cuda in discovering real-world vulnerabilities. Specifically, our test targets consist of four independent, previously unknown real-world software defects, all identified by NVIDIA PSIRT.

Given that AFL++, in its typical code coverage-based fuzzing paradigm, can not effectively obtain precise code coverage information for closed-source or highly optimized binary libraries through instrumented compilation or QEMU mode, we adopt vulnerability discovery time as its primary and core performance evaluation metric in this comparison. This methodology is particularly suitable for black-box testing scenarios, where the speed of vulnerability discovery becomes a critical measure of fuzzing efficiency when the underlying code structure is inaccessible.

The experimental results are presented in Table2. From the data, we observe that AFL++ demonstrated relatively

limited performance when tested against these five representative real-world vulnerabilities—it successfully detected only 2 vulnerabilities. The time AFL++ required to discover these vulnerabilities far exceeded that of Fuzz4Cuda, highlighting the latter's significant advantage in vulnerability discovery efficiency.

This performance gap can be attributed to inherent design limitations in AFL++. Since its core architecture primarily focuses on analyzing and manipulating CPU-side binaries, AFL++ lacks the capability to directly capture GPU-side error types such as memory overflows or data corruption. Consequently, it can not effectively detect and report complex vulnerabilities occurring within GPU compute cores or their memory management the domain where specialized GPU-oriented fuzzing tools like Fuzz4Cuda excel.

5.6. RQ4:Effectiveness of Persistent Mode

When fuzz testing GPU libraries, there is a big difference in execution speed between persistent mode and non-persistent mode. We gathered speed data from several harnesses using both modes, and the results clearly show that persistent mode leads to a remarkably obvious and immediate boost in efficiency.

As shown in Table3, the average execution speed increased by at least 50 times within Fuzz4Cuda. This acceleration proves the huge advantage of persistent mode in GPU fuzz testing.

Given the feature of GPU devices, persistent mode is very necessary for fuzz testing GPU libraries. The way GPU operations work and how they interact with the fuzzing process suggest that maintaining a persistent state significantly improves how effectively and efficiently vulnerabilities are found. In non-persistent mode, every fuzzing input means resetting the GPU state and reallocating resources. Persistent mode allow the fuzzing tool to keep the GPU active across many attempts. This avoids repeating initialization and cleanup. Using persistent mode is key to making the tests more efficient and effective.

5.7. RQ5:Effectiveness of Dynamic Breakpoints Quantity

To assess the influence of different speed drop thresholds on basic block coverage, we selected a specific test harness from each target library provided by the CUDA Toolkit to serve as a testing target. Each experiment was repeated 3

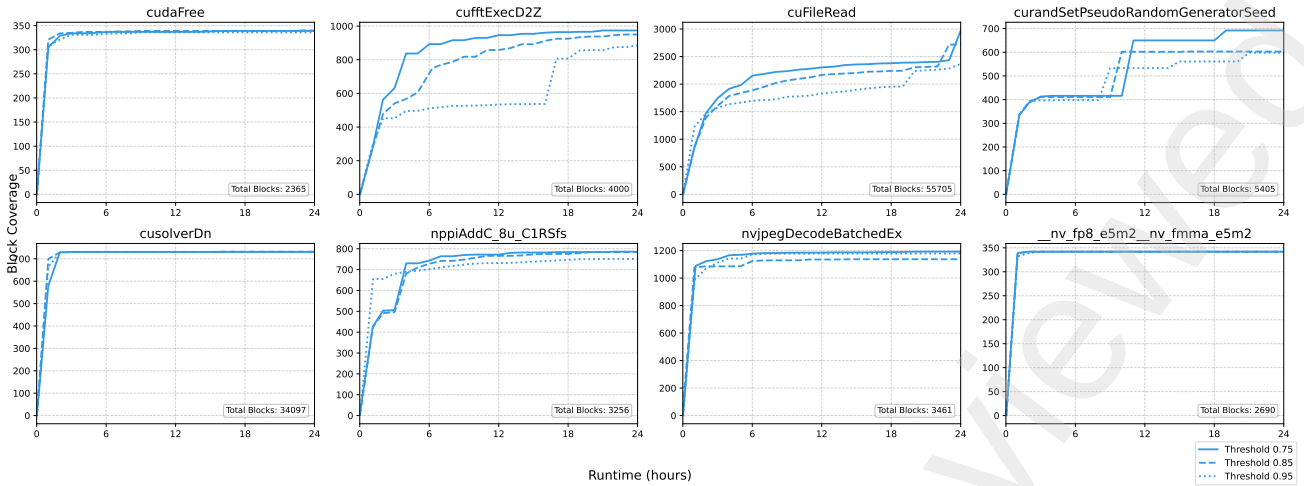


Figure 4: Block Coverage over 24 Hours for Eight Harnesses Across Different Libraries.

Table 3
Comparison of Persistent and Non-Persistent Execution Modes

Harness	Execution Speed	
	Persistent	Non-Persistent
cufftExecD2Z	11it/s	0.085it/s
cuFileRead	4.57it/s	0.082it/s
cudaFree	41.33it/s	0.092it/s
cusolverDn	27.31it/s	0.082it/s
nppiAdd	21.86it/s	0.082it/s
curandSetSeed*	47.68it/s	0.093it/s
nvjpegDecode*	20.85it/s	0.081it/s
nv_fp8_fmmda*	20.85it/s	0.080it/s

*curandSetSeed: curandSetPseudoRandomGeneratorSeed

*nvjpegDecode: nvjpegDecodeBatchedEx

*nv_fp8_fmmda: __nv_fp8_e5m2__nv_fmmda_e5m2

times to ensure the reliability and statistical significance of the results.

As Figure 4 illustrates, the test results demonstrate that our threshold drop method automates the setting of the number of breakpoints and controls the extent of performance degradation. This enables dynamic adjustment of fuzzing granularity based on actual operational needs, eliminating the requirement for manual intervention.

For nppiAddC_8u_C1RSfs and cuFileRead, the coverage in the early stages increased faster when the threshold was set to 0.95. We attribute this to the fact that a subset of basic blocks can be triggered by the majority of seeds, and with faster execution, breakpoint changes can be reached earlier, thereby covering these blocks sooner.

This method is especially suitable for scenarios involving a large number of basic blocks. When the program is complex and contains a vast number of basic blocks, manually setting breakpoints is both time-consuming and prone to errors. Our automated threshold drop method can handle these situations much more efficiently, ensuring that coverage is maintained while keeping the speed drop within

an acceptable range. This enhances the efficiency and practicality of fuzzing, especially when testing large, complex software libraries.

6. Discussion

The NVIDIA driver library and runtime library provide most of the functionalities for operating the GPU, ultimately invoking the underlying driver. If the graphics driver itself—currently distributed as a .so dynamic library—were available in source form and could be compiled into a static library, its control flow graph could be analyzed using reverse engineering tools such as Ghidra. In theory, this would make fuzzing the driver itself feasible, opening up new directions for GPU security research at a lower system level.

Moreover, while existing tools like Nvbit [28] are highly effective for GPU performance analysis and general-purpose instrumentation, they were not designed with fuzzing in mind. Similarly, NVBitFI [26] demonstrates the power of dynamic binary instrumentation for GPU reliability testing, yet its focus on fault injection rather than fuzzing leaves a gap in security-oriented dynamic analysis. We believe that integrating runtime feedback information from Nvbit into the fuzzing workflow holds significant potential. By leveraging Nvbit's instruction-level monitoring capabilities, it is possible to build a GPU fuzzing framework with substantially reduced initialization overhead and improved efficiency. This integration would eliminate the need to construct a dedicated fuzzing instrumentation layer from scratch, thereby lowering resource consumption and test execution time while preserving high code coverage and vulnerability discovery capabilities.

7. Conclusion

This work presents Fuzz4Cuda, a dedicated fuzzing framework targeting the security of CUDA-based GPU libraries. Unlike traditional fuzzers that operate on CPU architectures, Fuzz4Cuda addresses the long-standing challenge of collecting coverage and performing effective gray-box fuzzing on the GPU side. By leveraging real-time analysis of CUDA Streaming Assembler and employing dynamic breakpoint adjustment strategies, Fuzz4Cuda enables efficient, in-depth exploration of GPU kernel code.

Our month-long evaluation of the CUDA Toolkit demonstrated Fuzz4Cuda's practical effectiveness, resulting in the discovery of five real-world bugs—four of which have been assigned CVE identifiers. These results underscore the importance and feasibility of GPU-side fuzzing for enhancing the overall security of the GPU programming environment. Fuzz4Cuda represents a significant step forward in bridging the gap between fuzz testing and GPU software, opening new possibilities for future research and vulnerability discovery in heterogeneous computing systems.

8. Acknowledgements

This work is funded by the Military-wide Shared Information System Equipment Pre-research (Project No. 31511080306).

References

- [1] AMD GPUOpen, 2025. RDNA Architecture. <https://gpuopen.com/rdna/>. Accessed: 21 July 2025.
- [2] Bernhard, L., Schiller, N., Schloegel, M., Bars, N., Holz, T., 2024. Darthshader: Fuzzing webgpu shader translators & compilers, in: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, pp. 690–704.
- [3] Cao, S., He, B., Sun, X., Ouyang, Y., Zhang, C., Wu, X., Su, T., Bo, L., Li, B., Ma, C., et al., 2023. Oddfuzz: Discovering java deserialization vulnerabilities via structure-aware directed greybox fuzzing, in: 2023 IEEE Symposium on Security and Privacy (SP), IEEE. pp. 2726–2743.
- [4] Chen, P., Xie, Y., Lyu, Y., Wang, Y., Chen, H., 2023. Hopper: Interpretative fuzzing for libraries, in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, pp. 1600–1614.
- [5] Chesser, M., Nepal, S., Ranasinghe, D.C., 2023. Icicle: A re-designed emulator for grey-box firmware fuzzing, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 76–88.
- [6] Deng, Y., Xia, C.S., Peng, H., Yang, C., Zhang, L., 2023a. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models, in: Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis, pp. 423–435.
- [7] Deng, Y., Xia, C.S., Yang, C., Zhang, S.D., Yang, S., Zhang, L., 2023b. Large language models are edge-case fuzzers: Testing deep learning libraries via fuzzgpt. [arXiv preprint arXiv:2304.02014](https://arxiv.org/abs/2304.02014).
- [8] Deng, Y., Yang, C., Wei, A., Zhang, L., 2022. Fuzzing deep-learning libraries via automated relational api inference, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 44–56.
- [9] Eisele, M., Ebert, D., Huth, C., Zeller, A., 2023. Fuzzing embedded systems using debug interfaces, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 1031–1042.
- [10] Even-Mendoza, K., Sharma, A., Donaldson, A.F., Cadar, C., 2023. Gray: Greybox fuzzing of compilers and analysers for c, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 1219–1231.
- [11] Fioraldi, A., Mantovani, A., Maier, D., Balzarotti, D., 2023. Dissecting american fuzzy lop: a fuzzbench evaluation. *ACM transactions on software engineering and methodology* 32, 1–26.
- [12] Godefroid, P., Klarlund, N., Sen, K., 2005. Dart: Directed automated random testing, in: Proceedings of the 2005 ACM SIGPLAN conference on Programming language design and implementation, pp. 213–223.
- [13] Jeong, B., Jang, J., Yi, H., Moon, J., Kim, J., Jeon, I., Kim, T., Shim, W., Hwang, Y.H., 2023. Utopia: Automatic generation of fuzz driver using unit tests, in: 2023 IEEE Symposium on Security and Privacy (SP), IEEE. pp. 2676–2692.
- [14] Jiang, B., Wang, X., Chan, W.K., Tse, T., Li, N., Yin, Y., Zhang, Z., 2020. Cudasmith: A fuzzer for cuda compilers, in: 2020 IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC), IEEE. pp. 861–871.
- [15] Jung, J., Jang, J., Jo, Y., Vinck, J., Voulimeneas, A., Volckaert, S., Song, D., 2025. Moneta: Ex-vivo gpu driver fuzzing by recalling in-vivo execution states, in: Network and Distributed System Security (NDSS) Symposium 2025, Internet Society. Internet Society, San Diego, CA, USA. p. 230218. doi:10.14722/ndss.2025.230218.
- [16] Lee, M., Cha, S., Oh, H., 2023. Learning seed-adaptive mutation strategies for greybox fuzzing, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE. pp. 384–396.
- [17] Lidbury, C., Lascu, A., Chong, N., Donaldson, A.F., 2015. Many-core compiler fuzzing. *SIGPLAN Not.* 50, 65–76. URL: <https://doi.org/10.1145/2813885.2737986>, doi:10.1145/2813885.2737986.
- [18] LLVM Project, 2024. Libfuzzer documentation. URL: <https://rocm.docs.amd.com/projects/llvm-project/en/latest/LLVM/11vm/html/LibFuzzer.html>. accessed: 2025-06-27.
- [19] Maier, D., Toepfer, F., 2021. Bsod: Binary-only scalable fuzzing of device drivers, in: Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses, pp. 48–61.
- [20] NVIDIA Developer, 2024. CUDA Zone. URL: <https://developer.nvidia.com/cuda-zone>.
- [21] NVIDIA Developer, 2025a. NVIDIA CUDA Documentation. <https://docs.nvidia.com/cuda/>. Accessed: July 2, 2025.
- [22] NVIDIA Developer, 2025b. NVJPEG. <https://docs.nvidia.com/cuda/nvjpeg/index.html>. Accessed: July 2, 2025.
- [23] Peng, C., Rajan, A., 2020. Automated test generation for opengl kernels using fuzzing and constraint solving, in: Proceedings of the 13th Annual Workshop on General Purpose Processing using Graphics Processing Unit, pp. 61–70.
- [24] Peng, C., Zhang, X., Yu, G., Luo, G., Sun, J., 2017. Large kernel matters—improve semantic segmentation by global convolutional network, in: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 4353–4361.
- [25] Peng, X., Jia, P., Fan, X., Liu, J., 2025. Enzz: Effective n-gram coverage assisted fuzzing with nearest neighboring branch estimation. *Information and Software Technology* 177, 107582.
- [26] Tsai, T., Hari, S.K.S., Sullivan, M., Villa, O., Keckler, S.W., 2021. Nvbitfi: Dynamic fault injection for gpus, in: 2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). URL: <http://dx.doi.org/10.1109/dsn48987.2021.00041>, doi:10.1109/dsn48987.2021.00041.
- [27] Vikram, V., Laybourn, I., Li, A., Nair, N., O'Brien, K., Sanna, R., Padhye, R., 2023. Guiding greybox fuzzing with mutation testing, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 929–941.
- [28] Villa, O., Stephenson, M., Nellans, D., Keckler, S.W., 2019. Nvbit: A dynamic binary instrumentation framework for nvidia gpus, in: Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture, pp. 372–383.

- [29] Wang, Z., Yan, M., Chen, J., Liu, S., Zhang, D., 2020. Deep learning library testing via effective model generation, in: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 788–799.
- [30] Wei, A., Deng, Y., Yang, C., Zhang, L., 2022. Free lunch for testing: Fuzzing deep-learning libraries from open source, in: Proceedings of the 44th International Conference on Software Engineering, pp. 995–1007.
- [31] Xie, C., Jia, P., Yang, P., Hu, C., Kuang, H., Ye, G., Hong, X., 2023. Not all seeds are important: Fuzzing guided by untouched edges. *Applied Sciences* 13, 13172.
- [32] Xie, X., Ma, L., Juefei-Xu, F., Xue, M., Chen, H., Liu, Y., Zhao, J., Li, B., Yin, J., See, S., 2019. Deephunter: a coverage-guided fuzz testing framework for deep neural networks, in: Proceedings of the 28th ACM SIGSOFT international symposium on software testing and analysis, pp. 146–157.
- [33] Zhang, Z., Chen, L., Wei, H., Shi, G., Meng, D., 2024. Prospector: Boosting directed greybox fuzzing for large-scale target sets with iterative prioritization, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 1351–1363.
- [34] Zhu, J., Lin, M., Yin, T., Cai, Z., Wang, Y., Chang, R., Shen, W., 2024. Crossfire: Fuzzing macos cross-xpu memory on apple silicon, in: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, Association for Computing Machinery, New York, NY, USA. p. 3749–3762. URL: <https://doi.org/10.1145/3658644.3690376>, doi:10.1145/3658644.3690376.